

Project Report: Carré Surfer

Group 3

May 14, 2026

Note on Group Work: This project was initially started as a group of two. However, my partner decided to withdraw during the development process, citing insufficient time to complete both this project and assignments from other courses. Consequently, the entirety of the implementation and advanced features presented in this report were completed individually.

1 Introduction

As part of the INFO0004-2 project, I developed *Carré Surfer*, a C++ endless runner game. The player takes on the role of Thomas, a student walking through the "Carré" district in Liège, who must avoid various obstacles while trying to get back home.

The objective of this project was to put into practice the object-oriented programming concepts covered in class, including inheritance, polymorphism, and rigorous memory management. I also aimed to enhance the gaming experience by adding advanced features, including an autonomous artificial intelligence.

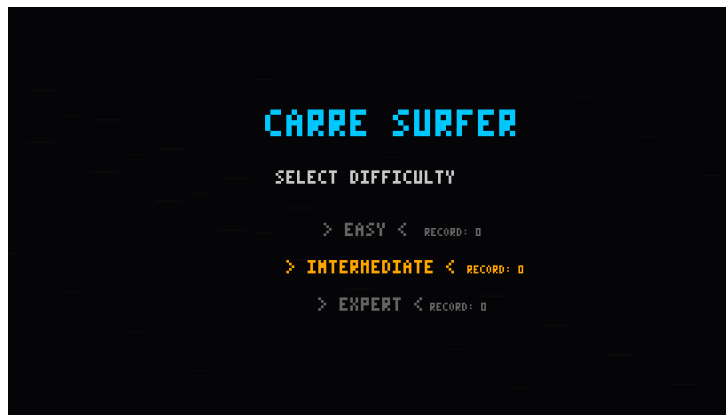


Figure 1: Game home screen showing the different difficulty modes.

2 Project Implementation

To meet the initial requirements of the project statement, I structured my code around several classes to manage Thomas and his environment in a modular way.

2.1 Architecture and Class Roles

I adopted an object-oriented design based on an entity hierarchy, as illustrated in the following diagram:

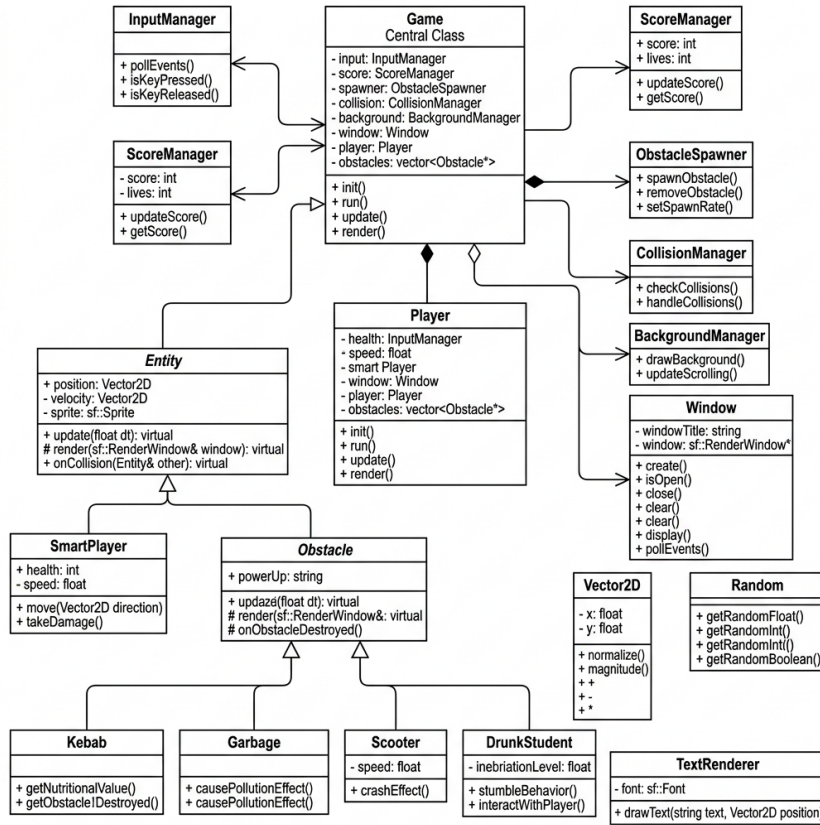


Figure 2: Class diagram representing the project architecture.

- **Entity**: Abstract base class defining common properties (position, size, velocity) and the virtual `update` and `draw` methods.
- **Player** and **Obstacle**: Specializations of **Entity**. Thomas and the various scrolling objects inherit from these classes to define their own behaviors.
- **Game**: Central class orchestrating the main loop and communication between different systems.
- **Management Systems**: Several specialized classes handle technical aspects: **InputManager** (keyboard/mouse), **ObstacleSpawner** (generation), **CollisionManager** (impact detection), **BackgroundManager** (parallax background), and **ScoreManager** (save management).
- **Utilities**: The **Vector2D**, **Random**, and **TextRenderer** classes provide cross-cutting tools for physics, randomness, and text display.

2.2 Mandatory Features

I implemented Thomas's movements as well as his "stagger" behavior (random zigzags). Health management is linked to screen brightness, and collisions trigger visual effects such as screen shake and temporary invincibility.



Figure 3: Thomas in-game avoiding obstacles.

3 Extensions and Personal Additions

Beyond the core features, I integrated several extensions to improve the overall quality of the project.

3.1 Text Rendering Engine (TextRenderer)

Since the Wayland library did not allow for direct text display, I developed a rendering engine (**TextRenderer**). It decomposes each character into a pixel matrix (3x5) represented by bitmasks in memory.

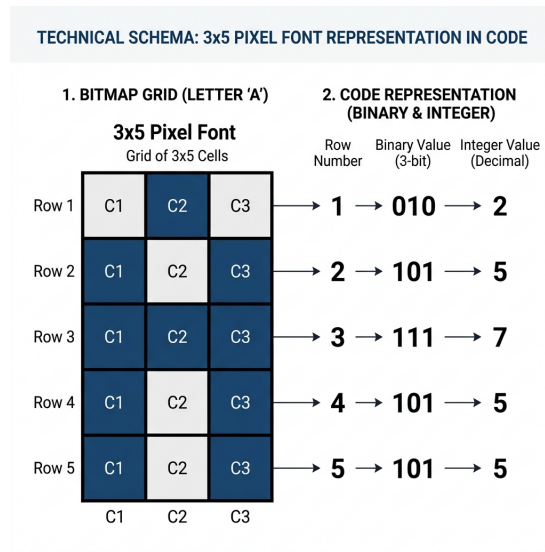


Figure 4: Matrix representation of a character in the **TextRenderer**.

3.2 SmartPlayer: Artificial Intelligence

Inspired by the INFO8006-1 (Artificial Intelligence) course, I created the **SmartPlayer** class. It inherits from **Player** and uses a predictive algorithm to analyze upcoming obstacles and optimize its trajectory in real-time. This mode can be activated by launching the game with the command `./game -smart`, allowing Thomas to play fully autonomously.

3.3 Persistent Score Management

I implemented a `ScoreManager` that saves records persistently. Scores are now differentiated according to the chosen difficulty level, providing better replayability.

3.4 Interface and Immersion

The addition of a smooth main menu, difficulty selection, and a background with a parallax effect contributes to a more immersive user experience.

4 Technical Challenges and Difficulties Encountered

Developing the core mechanics presented several challenges, particularly regarding entity synchronization and interaction precision.

4.1 Obstacle Generation Management

One of the main challenges was designing an `ObstacleSpawner` capable of generating obstacles at random intervals while maintaining consistent speed for all objects on screen. I had to ensure that obstacle density remained playable despite the randomness, by adjusting the appearance probabilities defined in the project statement (Kebab, Scooter, Student, etc.).

4.2 Collision Detection with CollisionManager

Implementing the `CollisionManager` required special attention. I used Axis-Aligned Bounding Boxes (AABB) for impact detection. The difficulty lay in the polymorphic handling of collisions: the manager must identify the impact, but it is up to each object (`Obstacle` or `Player`) to trigger its own behavior (`onCollision`) according to its nature (health loss, healing, or control inversion).

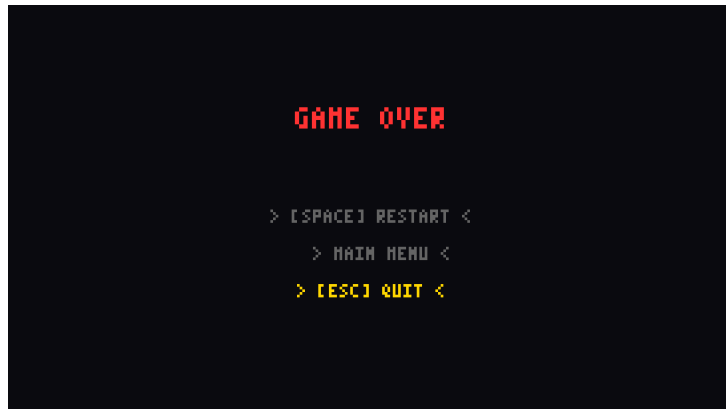


Figure 5: Game over interface showing the final score.

5 Conclusion

In conclusion, the realization of *Carré Surfer* has been an enriching experience that allowed me to consolidate my knowledge of object-oriented programming. The rigorous application of inheritance and encapsulation principles facilitated the integration of new features.

This project also taught me how to manage the constraints associated with developing a game engine from a low-level library, while ensuring the readability and robustness of the source code.

Finally, in accordance with the project guidelines, I specify that artificial intelligence was used occasionally to generate some missing code comments at the end of development, as well as for the creation of the submission script in the Makefile.